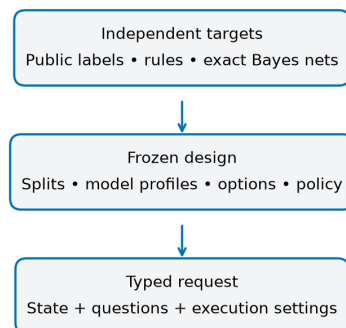
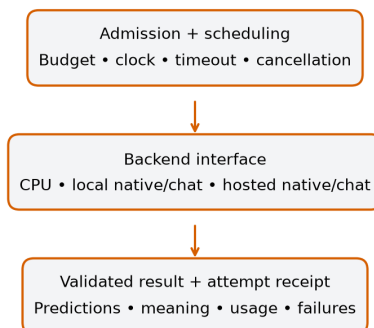


Modular decisions: independent targets, explicit execution, retained evidence

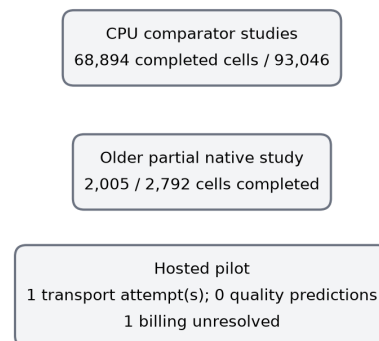
a Define the comparison



b Execute declared lanes



c Retain distinct evidence



Completion includes capability work. Cohorts and source identities differ; the full comparative study remains unfinished.

Modular Decision Models: Typed Interfaces, Orchestration, and Reproducible Evaluation

Evidence from CPU Baselines, Partial Local Model Studies, and a Hosted Capability Pilot

Daniel Ari Friedman

Active Inference Institute ORCID 0000-0001-6232-9096

Working manuscript | package version 0.6.0

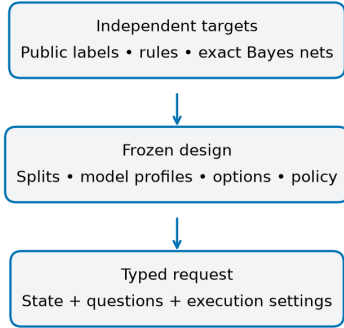
DOI 10.5281/zenodo.22816187

github.com/docxology/daf-jev

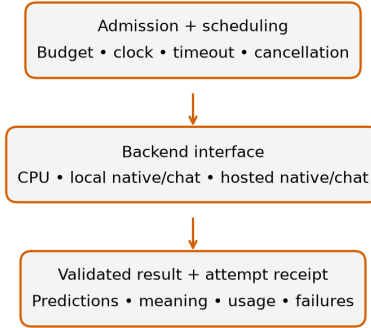
1 Abstract

Modular decisions: independent targets, explicit execution, retained evidence

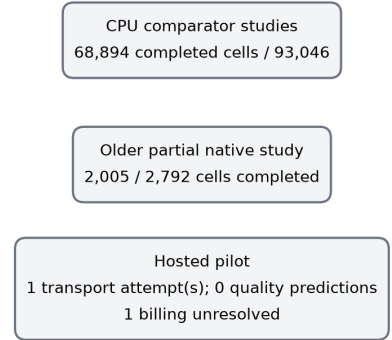
a Define the comparison



b Execute declared lanes



c Retain distinct evidence



Completion includes capability work. Cohorts and source identities differ; the full comparative study remains unfinished.

Figure 1: Graphical abstract of the modular decision-evaluation workflow. Left: independent public labels, explicit rules or a known Bayes reference define targets; frozen splits, model profiles, options and policies define the comparison; typed requests preserve states, questions and execution settings. Center: budget, cumulative clock, timeout and cancellation admission precede scheduling through the backend interface for CPU comparators, local native/chat runtimes and hosted native/chat endpoints. Validated predictions and per-attempt receipts retain probability meaning, usage and failures. Right: exact selected summaries report completed CPU cells, an older partial native study and the halted hosted pilot with unresolved billing. Completion includes capability work. The lanes use different cohorts and source identities, so the panels communicate evidence coverage rather than a common model-quality ranking; the full comparative study remains unfinished.

Software decisions require more than syntactically valid model output: a declared task, complete vocabulary, interpretable uncertainty, an evaluated policy and reliable execution. daf-jev provides typed native decision clients and pure composition, extended by provider adapters and a frozen benchmarking protocol for local models, hosted decision endpoints, constrained generation and supervised references. It retains output provenance and probability meaning rather than treating a generated label as a belief distribution.

The study distinguishes proper scoring against independent labels from distribution concentration and modal self-agreement. It supports grouped evaluation, validation-selected selective policies, cost-aware cascade replay, exact Bayes inference under supplied factors, and immutable per-attempt accounting. Unsupported tasks, unresolved outcomes and unknown charges remain visible in planned denominators.

Selected historical API receipts show workload-specific batching and pipeline behavior; their reliability diagram uses a repeat-agreement proxy and does not establish ground-truth calibration. Later CPU comparator studies contain 9 jobs with 68894 completed cells out of 93046 planned cells, with inapplicable combinations retained as unsupported. Their final-train tests, leakage-clean refits and training-fold validation have distinct interpretations. Partial native evidence remains separate. The hosted pilot made 1 failed capability attempt and retained 1 unknown-billing attempt, halting admission under the shared USD 25 allocation; it supplies no hosted quality comparison.

Current software coverage is 93.11%, bound to retained verification inputs. Generated variables, figures and prose share exact selected evidence, including the documentation snapshot 708902db9820d9d8. The contribution is a reusable decision-evaluation framework and an explicit account of what the selected experiments establish, rather than a universal provider ranking or deployment guarantee.

Keywords: decision models, LLM APIs, probability calibration, batching, confidence-gated routing, Python client, reproducible research

2 Introduction: From Generated Text to Typed Decisions

Applications often need a bounded judgment at a particular point in their control flow: classify an intent, estimate whether a condition holds, or assign a rubric score. Free-form answers require an interpretation layer, but modern generative systems can

also constrain their output to a grammar or JSON schema [Willard and Louf, 2023, OpenRouter, 2026b]. The research problem therefore extends beyond parsing. A valid output must still refer to the right task, preserve its complete answer vocabulary, communicate only the uncertainty it actually supports, and enter a policy whose quality and cost can be evaluated.

TypeSafe describes Jev as a System One model returning typed judgments and distributions, with reinforcement learning for calibrated decisions (RLCD) as a training objective [TypeSafe AI, 2026e,f]. That documented objective motivates an interface; it does not establish calibration on a new dataset. A generated label, a normalized compatibility distribution, a supervised class-probability estimate, and an analytical conditional probability are distinct outputs even when they share a JSON shape. This manuscript treats their origin, declared meaning and empirical validation as separate parts of the decision contract.

2.1 The problem this package solves

The initial package implemented the native System One surface: typed questions, response validation, transport errors and pure composition. Its extension addresses a broader methodological need: compare local open-source runtimes, hosted decision endpoints, constrained chat models and fixed supervised references without silently making their outputs or experimental conditions equivalent. An orchestration layer must retain unsupported tasks and failed attempts, distinguish selection from evaluation, and account for unresolved costs before admitting more requests.

daf-jev provides four connected components:

- **Typed primitives:** ergonomic builders (`noul()`, `choice()`, `score()`) and a `QuestionSet` container, with strict request and response validation.
- **Provider adapters:** synchronous and asynchronous native clients, plus a benchmark backend boundary for native decisions, constrained generated values and local references. Capabilities and probability meanings remain explicit.
- **Composable policies:** pure functions over answers (`composite_score`, `confidence_gate`, `route`, `pick`), validation-selected gates and cascade replay. Policy selection and executed policy timing have separate evidence requirements.
- **A reproducible evaluation protocol:** frozen dataset views, question and option order, source/runtime identities, per-attempt receipts, planned denominators, cost admission and offline reports. Real HTTP and owned-process fixtures test failure paths without substituting a fixture result for model execution.

The contribution is a modular implementation and an auditable study design, rather than a new learning algorithm or a claim that one provider is universally superior. Following the multi-axis evaluation motivation of HELM [Liang et al., 2023], the protocol exposes quality, probability interpretation, selective coverage, reliability, latency and cost together. The selected evidence is heterogeneous: historical hosted measurements, completed CPU comparator studies, partial native execution and a failed hosted capability probe. Their boundaries are part of the results, not omissions to be concealed by reporting only successful predictions.

2.2 Reader’s guide

3 distinguishes native decision contracts from constrained generation and discusses probability semantics. 4 defines the modular implementation, metrics, selective policies and exact graphical inference. 5 describes its software interfaces. 6 separates historical observations from current selected comparator and execution evidence. 7 records datasets, splits, controls and measurement units; 8 explains exact evidence selection and regeneration. 9 relates the approach to forecasting, selective prediction, cost-aware cascades and benchmark-overlap research, and identifies the remaining limits.

3 Decision Interfaces: Native Judgments, Generated Values, and Probability Meaning

The original native interface is documented by TypeSafe’s retained System One snapshot. Its statements about training, architecture and performance are vendor claims, distinguished here from the package’s validation rules and selected measurements. The wider benchmark interface also supports generated values and supervised references; compatibility with that interface does not establish identical uncertainty semantics.

3.1 The native decision contract

TypeSafe positions Jev as a component supplying narrow judgments to application-owned control flow [TypeSafe AI, 2026e]. Its documented RLCD objective aims at calibrated decisions [TypeSafe AI, 2026f]. The manuscript does not infer achieved calibration, correctness or deployment suitability from that objective. Third-party launch coverage remains historical commentary, rather than evidence for a model’s present behavior [Claburn, 2026, Corvin, 2026].

A native request carries a state and caller-keyed questions. The documented primitives are [TypeSafe AI, 2026a,d]:

- **noul**: a declared probability for a binary condition, optionally described by contrasting true/false criteria.
- **choice**: a selected label, a distribution over the complete named vocabulary, and a scalar confidence field.
- **score**: a possibly fractional rating over ordered level indices, with its level legend, distribution and confidence field.

Native clients validate response shape, finite numeric values and question compatibility. Benchmark adapters retain available usage and request identifiers in attempt receipts, including when later semantic parsing fails. These checks establish an admissible response under the stated contract; they do not establish that the selected label is correct. Missing usage, resolved provider or billing fields remain unavailable rather than being inferred from a successful status code.

3.2 Documented isolation and backend capacity

TypeSafe describes questions as independently evaluated within a parallel request [TypeSafe AI, 2026e,c]. That supports the documented batching pattern, but independence is a provider claim to test when comparing actual runtimes. The package can express isolated questions without proving a backend’s internal computation is isolated. A paired experiment with identical state, questions and options is needed to assess whether batching changes predictions.

Capacity is profile-specific: question count, complete option vocabulary, input size and supported primitive types must all fit. A broad catalog context field alone does not establish how a native server tokenizes multiple questions or bills their aggregate input. Oversized or uncertain inputs remain planned unsupported cells; the evaluation does not shorten states, drop rare classes or substitute another model to improve completion.

3.3 Constrained generation is a separate instrument

Guided decoding can restrict a language model’s output to a formal language [Willard and Louf, 2023]. The chat adapter supplies a strict per-request JSON schema containing all required question identifiers, full choice enumerations and numeric bounds. Official OpenRouter documentation describes structured outputs, while warning that support and enforcement depend on the selected endpoint [OpenRouter, 2026b]. The adapter therefore validates returned values itself.

A generated label or bounded scalar remains a generated value. No one-hot belief or confidence is manufactured from it. Token likelihoods, where available, concern token sequences and need an independently justified mapping before they can be treated as probabilities over decision outcomes. Native Decisions and generated chat also use distinct documented endpoints [OpenRouter, 2026a]; catalog discovery and documentation availability do not prove a particular authenticated request will succeed.

3.4 Probability validity, concentration and calibration

Numerical validity asks whether a distribution is finite, nonnegative and has the permitted total mass. Concentration asks how strongly that distribution favors particular outcomes. Calibration concerns its relationship to observed outcomes; a concentrated but systematically wrong forecast can be valid and poorly calibrated. Forecasting theory distinguishes calibration from sharpness and motivates evaluating both with proper scoring rules [Gneiting and Raftery, 2007].

The benchmark retains declared meanings such as training-label frequency, estimated class probability and analytical conditional probability. A native row with undocumented meaning stays unknown. Its distribution can still be compared descriptively with a labeled target, but a numeric score does not convert compatibility weights into established posterior beliefs. Provider confidence fields and entropy-derived statistics likewise do not automatically estimate the probability that a selected decision is correct [TypeSafe AI, 2026b, Guo et al., 2017].

Historical self-agreement is narrower still. Repeating a wrong answer consistently produces a stable modal label. Agreement with that label measures repeat behavior, not correctness against an independent target. The historical reliability diagram in 6 is explicitly a proxy-agreement diagram. Application-owned gates require labeled validation for their actual loss and coverage objective, followed by evaluation of the frozen policy.

4 Methodology: The Layered Architecture of a Composable Decision Toolkit

daf-jev is organized as a strict layering: an ergonomic surface (CLI, MCP server, scripts, benchmarks) on top of pure-logic composition, on top of primitives and wire types, on top of explicit transport and backend boundaries. The core native client and the additive benchmark adapters retain separate request lifecycles. 2 shows the module graph. The layering is enforced by convention: the composition and primitive modules import no I/O machinery, so every decision pattern is testable against

constructed answers without a network stand-in. The ergonomic surface itself — CLI subcommands, MCP tools, an agent skill, and worked examples — is the subject of 5; this section covers the layers those surfaces delegate to.

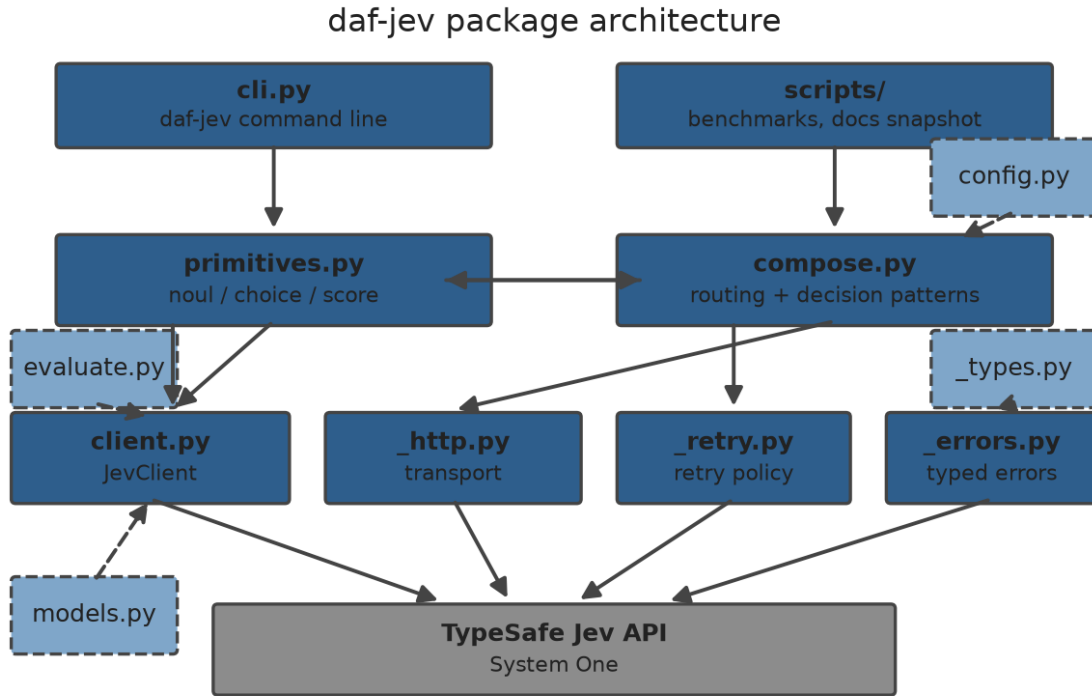


Figure 2: Package layer diagram for daf-jev, drawn from the module graph of `src/daf_jev/`. The application layer — the CLI (`src/daf_jev/cli.py`) and the thin orchestration scripts (`scripts/`, `benchmarks/`) — delegates downward to the logic layer, where `compose.py` (pure decision patterns) and `primitives.py` (typed question builders and the `QuestionSet` container) sit over the wire dataclasses of `_types.py`. `client.py` and `_http.py` implement the core native transport, with `_retry.py` providing the pure retry policy and `_errors.py` the typed exception hierarchy; `models.py` and `config.py` enter as side inputs, and `evaluate.py` is the batch harness. The key structural takeaway is the enforced dependency direction: composition and primitives import no I/O machinery, so every decision pattern is testable against constructed answers without a network stand-in. Boxes are named modules only; no measured values appear in this schematic.

4.1 Layered architecture

The layers, bottom-up:

- **Transport** (`_http.py`) — a minimal Transport protocol with a synchronous and an asynchronous `httpx` implementation. It knows nothing about decisions; it posts JSON to a path and returns the raw response.
- **Client** (`client.py`) — `JevClient` / `AsyncJevClient` resolve the API key from the environment (injected mapping, process environment, then a `.env` file, in that precedence order via `config.py`), build the request for a state plus a mapping of questions, apply the retry policy, and map HTTP status classes onto the typed exception hierarchy of `_errors.py`. The retry policy itself (`_retry.py`) is a *pure* function of the attempt number and an optional server-supplied retry hint — exponential backoff with a base, a cap, and jitter, with the hint winning when present — so its timing behavior is unit-testable without sleeping.
- **Wire types** (`_types.py`) — frozen dataclasses for the three question types and the three answer types, a strict parser that rejects unknown answer shapes, a usage record, and the top-level response object with cached per-type views (`nouls`, `choices`, `scores`).
- **Primitives** (`primitives.py`) — the ergonomic builders `noul()`, `choice()`, `score()` and the `QuestionSet` mapping container with additive composition (`add`, `merge`, `to_wire`). No I/O.
- **Composition** (`compose.py`) — pure functions over answers, described below. No I/O; network access happens only through a client injected by the caller for multi-call helpers.
- **Harness** (`evaluate.py`) — the batch `Evaluator`, described below.

4.2 Typed primitives

Jev question primitives and typed answers

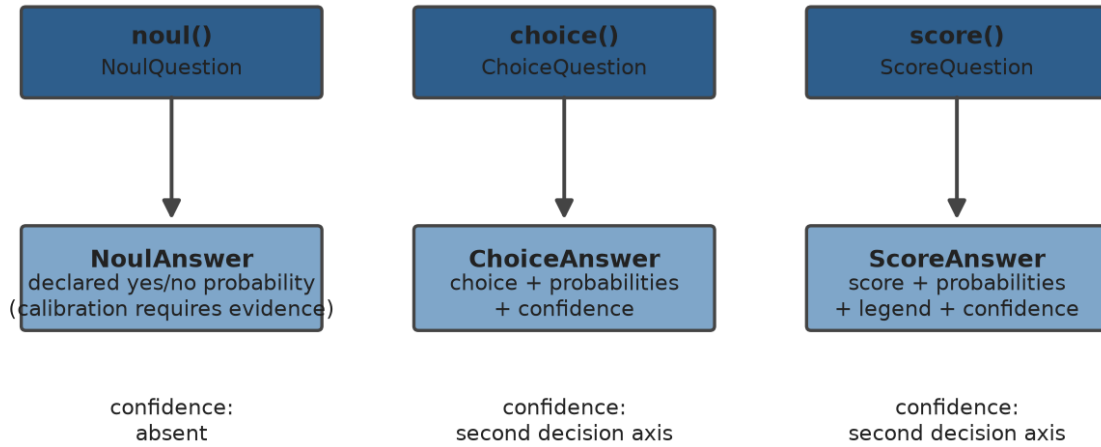


Figure 3: The three question primitives of the System One surface and the typed answer shapes `daf-jev` parses them into, drawn as a schematic of the wire contract (3). A `noul` question yields a declared yes/no probability; a `choice` question yields a selected label, a full probability distribution over the named options, and a scalar confidence; a `score` question yields a probability-weighted score over ordered levels together with the level legend, the level distribution, and a confidence. The takeaway is that the builders in `primitives.py` map one-to-one onto these shapes with strict, eager validation — score criteria must carry at least two ordered levels, choice criteria must be non-empty, and the answer-side parser rejects any payload outside the discriminated union — so downstream composition code can pattern-match on answers without defensive parsing. The diagram carries no measured data.

The builders in 3 map one-to-one onto the wire contract of 3. Validation is strict and eager: score criteria must carry at least two ordered levels, choice criteria must be non-empty, and the answer-side parser rejects unsupported types and malformed required answer fields. Builders validate requests, and the runtime parser validates received answers before downstream composition. This removes shape guessing without turning a valid answer into a correctness guarantee.

4.3 Composable decision patterns

The composition layer implements the documented TypeSafe patterns as pure functions [TypeSafe AI, 2026c]:

- **composite_score(answer, weights=None)** — the expected level index under the supplied distribution, optionally reweighted by finite caller-supplied weights with positive total weight and nonzero weighted mass. Negative weights are deliberately allowed but remove the bounded-index guarantee; reweighting is not a learned utility model. This turns an ordinal rating into a comparable scalar without a second model call.
- **confidence_gate(answer, threshold, below)** — returns the answer’s primary value when its confidence meets the caller’s threshold, and the below verdict otherwise. The threshold is owned by the calling code, never by the package.
- **route(answer, handlers, min_confidence, fallback)** — dispatches to the handler registered for a choice answer’s label, gated on a minimum confidence with an optional fallback handler; pick fans the same dispatch across every answer in a mapping.

Batching fan-out and the Evaluator. Under the documented native parallel-question contract (3), batching a fixed question set can amortize transport/state overhead within backend capacity. It does not by itself prove decision equivalence or minimum monetary cost. The Evaluator in `evaluate.py` inverts the axis: it holds the question set fixed and runs it over *many states*, concurrently — through a thread pool for the synchronous client or `asyncio` with a semaphore for the asynchronous client. Per-state failures never abort the batch; they are captured into the evaluation record alongside the answers and the measured latency,

so one malformed state remains an explicit failure rather than discarding the whole batch. The historical benchmark scripts drive clients directly; the application Evaluator is a distinct convenience harness. Its default async batch lifecycle closes the client at batch end.

Confidence-gated routing. 4 illustrates the three-band policy that `confidence_gate` and `route` implement: answers whose confidence clears the upper band are automated, mid-band answers are routed to review, and answers below the lower band are escalated. The boundary markers in the figure are *example thresholds* — the semantics are the bands, not any particular cut points, which remain policy owned by the caller.

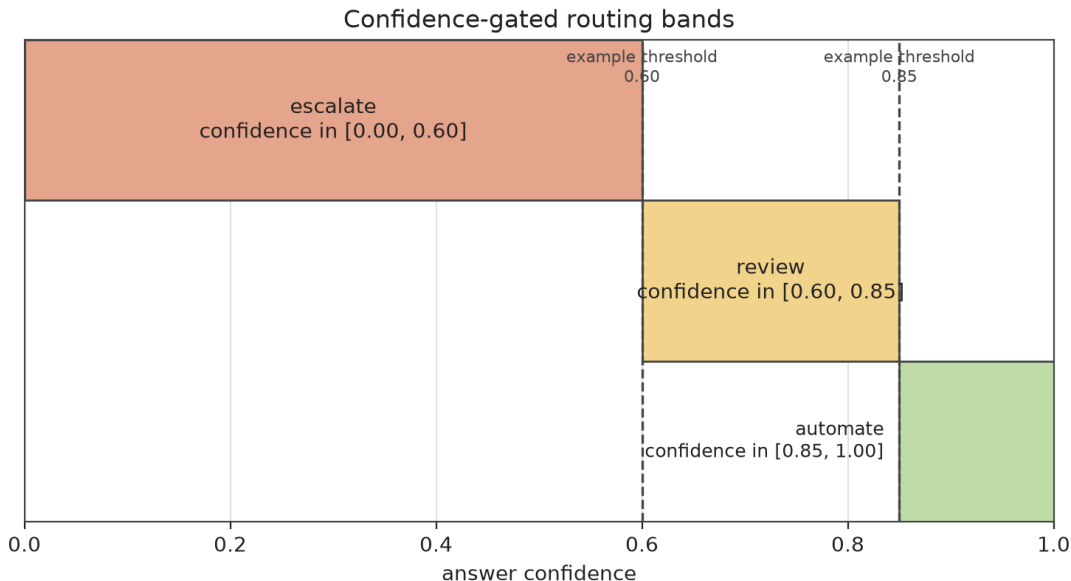


Figure 4: Parametric illustration of confidence-gated routing as implemented by `confidence_gate()` and `route()` in `src/daf_jeu/compose.py` (4). The horizontal axis is the model-reported confidence on the unit interval; the three horizontal bands assign an action per confidence region — automate above the upper boundary, route to review in the middle band, escalate below the lower boundary. The boundary markers are annotated as example thresholds: the semantics are the bands, not any particular cut points, and the actual thresholds remain policy owned by the caller. This figure is parametric and carries no measured data; 6 reports end-to-end latency and a self-agreement proxy; domain safety and correctness calibration require independent labeled validation.

4.4 Design rulings

Four rulings shaped the codebase and are worth stating as contracts:

1. **Pure logic, injected I/O.** Everything under `compose.py`, `primitives.py`, `_types.py`, and `_retry.py` performs no I/O; network access uses explicit clients or backend adapters, with parameterized transport and lifecycle seams for deterministic tests.
2. **Real stand-ins, no mocks.** The test suite drives the real HTTP transport against a real local HTTP server fixture rather than patching client internals, so retry, timeout, and error-mapping behavior is exercised end to end.
3. **Pure retry policy.** The configured attempt, retry hint, cap and jitter determine a delay without sleeping; disabling jitter gives deterministic timing tests, and both native clients use the same policy.
4. **Snapshotted documentation.** The bundled documentation snapshot carries a manifest of per-page content hashes and a snapshot identifier, and a CLI subcommand re-hashes the tree to detect drift (8). Prose claims about the model are therefore checkable against an immutable, verified source rather than a moving website.

4.5 Backend, task and policy abstractions

The benchmark backend receives a state and frozen typed questions, and returns predictions plus an attempt receipt. A profile declares primitive support, option and input limits, authentication scope, output provenance and probability meaning. The runner combines a backend profile, dataset view and protocol into immutable cells. A cell is the scheduling unit; it can contain multiple question-level decisions and multiple actual requests, as in a graphical experiment or cascade. These units must not be substituted for one another when reporting coverage or cost.

Native distributions are validated under their declared precision, without silent renormalization. Label-only outputs remain label-only. The deterministic rule reference reads authoritative synthetic state fields rather than target metadata; its ordinary labels acquire no invented beliefs. An analytical Bayes reference can return a genuine conditional distribution with analytical provenance. Fixed class-frequency and fitted supervised baselines declare their own probability meanings. Unsupported task/backend pairs remain in the planned matrix.

4.6 Quality metrics and uncertainty

Hard-label classification uses accuracy and macro F1 over the complete declared vocabulary. Out-of-scope detection is also reported separately when the task contains an explicit OOS class. An ordinal rubric has two relevant readings: a selected category and a scalar level index. Classification accuracy uses a supplied distribution’s argmax where available; MAE and RMSE evaluate the raw scalar in bin-index units. Averaging indices presumes a spacing convention and does not establish application utility or distance on the original physical measurement scale.

For a genuine distribution p and hard label y , multiclass Brier loss is

$$B(p, y) = \sum_{k=1}^K (p_k - \mathbf{1}\{y = k\})^2.$$

For a binary event, the separately named binary Brier loss is $(p_{\text{true}} - y)^2$. The multiclass expression is twice the binary expression for an exact two-class simplex. This convention must be declared when comparing reported scores. Brier loss descends from probabilistic forecast verification [Brier, 1950]; strict propriety provides an incentive to report the true distribution in expectation under its assumptions [Gneiting and Raftery, 2007]. It does not guarantee that a particular model has learned that distribution. The implementation also evaluates natural-logarithmic loss in nats, retaining infinite loss when a supplied probability assigns zero mass to a realized target rather than silently clipping it.

Reliability tables and expected calibration error summarize confidence/outcome correspondence using declared bins [Guo et al., 2017]. Binning, sample size and the correctness target affect that summary. In the historical benchmark, the target is modal repeat agreement. In supervised evaluation it is an independent label. Those quantities have different interpretations even when the same formula is applied. Missing probabilities or confidence yield unavailable probability metrics, not fabricated zeros.

Uncertainty intervals resample input groups, keeping repeated or matched physical rows together. The selected CPU intervals use retained grouped percentile draws on fixed predictions. They describe resampling variation for that evaluated prediction set; they do not include model refitting, shared-training dependence across folds, model-selection uncertainty or a future deployment shift. Overlapping training sets make cross-validation errors dependent [Bengio and Grandvalet, 2004]. The report therefore avoids treating pooled out-of-fold intervals as a universal variance estimate for the learning procedure.

4.7 Selective prediction and cascades

Let $a_\tau(x)$ indicate acceptance at confidence threshold τ , and let $\ell(\hat{y}, y)$ be decision loss. Selective coverage and risk are

$$C(\tau) = \mathbb{E}[a_\tau(X)], \quad R(\tau) = \frac{\mathbb{E}[a_\tau(X)\ell(\hat{Y}, Y)]}{C(\tau)}$$

when coverage is positive. This separates how often a policy answers from how often its accepted answers are wrong. Rejection without a defined fallback is not a correct decision. Selective classification research supplies algorithms with risk guarantees under specified sampling and selection assumptions [Geifman and El-Yaniv, 2017]; those guarantees do not follow from a generic confidence cutoff.

Here, candidate thresholds are selected on validation groups using a recorded Wilson upper bound, which requires sufficient groups for the selected risk target. An accepted group is correct only when all its accepted decisions are correct. Scanning candidate thresholds on the same validation data makes the retained bound a descriptive selection rule, not a simultaneous distribution-free risk guarantee. A gate with no eligible threshold accepts nothing and has unavailable accepted risk. Test evaluation applies the frozen selected policy; a final refit changes the underlying model and cannot inherit a prior gate without new validation.

A weak/strong cascade invokes the strong backend only where the weak policy rejects. Cost-aware cascade research motivates joint quality and expenditure objectives [Chen et al., 2024], but a route’s advantage must be measured under its actual invocation rule. Offline replay uses retained weak and strong predictions to score a counterfactual policy. It does not establish executed

cascade latency, serving contention, batching behavior or actual avoided charges. Executed cascades retain each child attempt, including partial failure or budget rejection after the weak call.

4.8 Exact graphical inference and elicited factors

The graphical layer represents finite discrete variables, a directed acyclic graph and complete conditional probability tables. Exact variable elimination computes

$$P(q \mid e) = \frac{\sum_z \prod_i P(x_i \mid \text{pa}_i)}{P(e)},$$

where the numerator fixes query and evidence assignments and sums hidden variables. Zero-probability evidence has no conditional distribution and raises an error. Exactness concerns the calculation under the supplied factorization, not whether the graph or factors describe the world. Elimination order and induced width can make computation expensive even for a modest number of variables [Dechter, 1999].

Async model-injected elicitation asks for every conditional row and can propose edges under a bounded search objective. The reference experiment discloses a known generative Bayes model and independently samples a hidden assignment. Returned beliefs are interpreted as normalized surrogate factors under that reference protocol; factor Brier error and posterior discrepancies are diagnostic comparisons, not evidence of learned causal structure or calibrated empirical CPTs. A structure proposal remains a graph proposal, without causal identification.

Evidence acquisition reveals an unobserved variable selected by its current marginal entropy, conditions the coupled posterior, and records the assignment, trajectory and fixed observation cost. Maximum current entropy is a heuristic; expected information gain asks how possible observations change uncertainty, and decision value additionally depends on downstream loss and cost [MacKay, 1992]. The implemented rule does not optimize either objective. Optional model re-asks after each reveal are separately receipted, and a backend lacking genuine belief rows cannot be converted to a graphical factor by inventing one-hot probabilities.

4.9 Reliability and cost accounting

Reports retain completed, failed, unsupported, unresolved and unattempted cells. A started cell without a terminal outcome is unresolved. Planned coverage uses the entire frozen obligation; attempted-outcome coverage describes the observed subset. Quality among successful predictions is consequently accompanied by the planned status distribution, so a difficult or unsupported arm cannot disappear from the denominator.

Token counts, reported API charges, reserved liability and local economic cost are different measurements. Known API charges are summed exactly, with unknown charge preserved as unknown. Admission requires a conservative sourced liability; a native catalog context length alone is insufficient to bound aggregate billable input across questions. A nominally free model does not make a response with missing billing metadata reconciled. Local hardware, energy and setup cost require a separately declared rate or measurement, and cannot be inferred from absent API billing.

Resource guards, cumulative allowances and owned cleanup constrain execution rather than model quality. A cancellation receipt can prove an attempt was interrupted without proving a final model outcome or charge. Raw unresolved windows remain unresolved; an additive nonrefundable allocation convention, if separately approved, must not be described as exact elapsed-time recovery or a rigorous upper bound on an unknown descendant. This distinction allows continuation designs to retain their uncertainty honestly.

5 Integration Surfaces: CLI, MCP Server, Agent Skill, and Worked Examples

The Python API is the primary door into daf-jev, but not the only one. Four additional surfaces sit on top of the same layering described in 4 — transport, client, primitives, composition — and add accessibility rather than logic: the command-line interface, an MCP server, an agent skill, and a set of runnable examples. These interfaces delegate policy and validation to the package modules; the benchmark orchestration surface additionally manages explicit planning, execution and reporting lifecycles. A behaviour fixed in the composition layer therefore reaches every consumer at once: the CLI and the MCP server call the same client and compose functions the Python API calls, the skill documents those entry points, and the examples exercise them.

5.1 Command-line interface

The `daf-jev` console script (`src/daf_jev/cli.py`) exposes commands that cover the package's main paths without writing Python:

- **`daf-jev ask`** — one call, one state: the state arrives as inline text or JSON (`--state`) or from a file (`--state-file`), and one or more `--question ID=SPEC` flags build the question set from a compact grammar (`noul:<instructions>`, `choice:<instructions>;k1=desc,k2=...`, `score:<instructions>;l1,l2,...`). The answer is emitted as JSON (compact by default, `--pretty` for readability).
- **`daf-jev evaluate`** — the batch Evaluator of 4 as a command: a fixed question set runs concurrently over many states, and the evaluation records — answers, per-state failures, and measured latency — are emitted as JSON.
- **`daf-jev models`** — lists the available model surface, with `--pick latest|first|last` selecting a single identifier for scripting.
- **`daf-jev docs-verify`** — re-hashes the bundled documentation snapshot against its manifest and exits non-zero on drift; this is the command-line face of Design ruling 4 (8).
- **`daf-jev serve`** — launches the MCP server described next, over the stdio transport.
- **`daf-jev providers`** — lists the stable registry without network access.
- **`daf-jev posteriors-load / posteriors-reask`** — validate retained sidecars and construct a max-entropy plan without a model call.
- **`daf-jev benchmark`** — prepares/fetches datasets, explicitly snapshots a public catalog, freezes a plan, executes/resumes and reports exact retained observations; its orchestration is documented in `docs/decision_benchmarking.md`.

The CLI is a thin adapter in the same sense as the composition layer is pure: it parses arguments, delegates to the client or the composition helpers, and serializes the result. It owns no retry policy, no parsing of answers, and no thresholds.

5.2 MCP server

`src/daf_jev/mcp_server.py` exposes the same core to any MCP-capable agent host. Launched with `daf-jev serve` over the stdio transport, it advertises nine tools that mirror the package one-to-one:

- `jev_ask` — drives the client's single-call path: named questions (`noul / choice / score`, given in the same compact grammar the CLI accepts or as native `{type, instructions, criteria}` dicts) asked about one state;
- `jev_evaluate` — drives the batch path, running a fixed question set over many states concurrently;
- `jev_models` — reports (and optionally picks from) the available model surface;
- `jev_composite_score`, `jev_confidence_gate`, and `jev_tiered_gate` — expose the composition patterns of 4; these combine answers locally with no API call at all;
- `jev_docs_verify` — re-runs the documentation drift check of Design ruling 4;
- `jev_posteriors_load` and `jev_reask_plan` — validate sidecars and compute an evidence-bearing posterior plan, with no API call.

A `jev://docs/snapshot` resource serves a summary of the bundled documentation snapshot itself, so an agent can inspect what the model-level claims rest on without leaving its host. The server is a thin adapter — it owns no retry policy, no parsing, and no thresholds; every call delegates to `JevClient`, `AsyncJevClient`, or the pure composition functions.

Registering the server with an MCP client is a one-block stdio configuration; with the package installed (or, equivalently, command: "uv" with args: ["run", "daf-jev", "serve"] inside the repository):

```
{
  "mcpServers": {
    "daf-jev": {
      "command": "daf-jev",
      "args": ["serve"]
    }
  }
}
```

The MCP extra (`pip install "daf-jev[mcp]"` or `uv sync --extra mcp`) is the only optional dependency this surface adds; the core package does not depend on it.

5.3 Agent skill

`skills/daf-jev/SKILL.md` is a skill manifest for coding agents: it states when to reach for the package, and how — the builders and their constraints, the client surface, the evaluation harness, the composition patterns, the CLI subcommands, and the MCP tools — so an agent can use the package correctly without reading its source. The skill carries no logic of its own; it is documentation positioned where agents look for it, and it points at the same entry points the CLI and MCP server expose.

5.4 Worked examples

Thirteen self-contained scripts under `examples/` walk the typical integration path from a first call to a batch evaluation, doubling as executable documentation of the composition layer:

- **`quickstart.py`** — the shortest path to a typed answer: build a question set, make one call, read the typed answers;
- **`triage_router.py`** — confidence-gated routing end to end: a choice call dispatched by `route()` with a fallback for low-confidence answers;
- **`composite_scoring.py`** — a score call turned into a comparable scalar by `composite_score()` and verdict-banded by `confidence_gate()`;
- **`evaluate_corpus.py`** — the batch Evaluator over many states, with per-state failure capture and summary aggregates;
- **`decider_loop.py`** — the decision-point loop end to end: observe → compose → ask → gate → fail-open, with typed hooks, a deterministic lexicon as the floor action, a Budget bound, and JSON decision receipts;
- **`gated_fallback.py`** — a deterministic keyword heuristic answering first, with the model called only when the heuristic is not confident enough, `confidence_gate(below="escalate")` as the final escalation lane, and a `UsageLedger` accounting every model call the fallback makes.

The remaining walkthroughs cover provider dispatch, Asia elicitation/inference, resilience, in-loop async evaluation, calibration, retry policies and posterior re-asking. Model-backed scripts use the real endpoint when a key is present and skip without a key; `providers_example.py` always uses an injected transport. These walkthroughs demonstrate APIs rather than a labeled benchmark corpus.

5.5 Benchmark orchestration and offline reports

The benchmark CLI exposes dataset preparation and views, explicit catalog snapshots, immutable planning, execution/resume and offline reporting. Planning performs no inference. Catalog retrieval is a separate public metadata operation. Execution requires the selected backend's actual capability, financial and resource admission; a saved plan does not grant those conditions. Reports retain dataset IDs and exact prepared-input hashes, planned cell and decision counts, probability-meaning availability and attempt outcomes.

This surface connects reusable adapters to experiments without placing policy inside a provider client. CPU comparators fit only the declared training view, gates select on validation, and the evaluator scores retained test targets. Markdown, JSON and PDF summaries can be regenerated from selected records without making a new model request. The example scripts demonstrate APIs; the experiment manifests and their retained receipts establish which benchmark operations actually ran.

6 Results: Historical Measurements, CPU Comparators, and Execution Coverage

This section separates historical native API measurements from later CPU comparator studies, partial local-model execution and a halted hosted probe. The historical batching benchmark implements the documented parallel-questions pattern [TypeSafe AI, 2026c], alongside decision-pattern latency and self-consistency measurements. These historical receipts identify their requested model aliases, rather than a pinned weight identity. Every value below is injected from the explicitly selected benchmark outputs at render time; figures must use that same evidence selection. The batching and latency results reported here were recorded on 2026-09-16; the calibration results on 2026-09-16.

6.1 Historical batching speedup and total-token use

The batching benchmark compares two strategies over the same state and question mix: one call carrying all questions in a batch, versus one sequential single-question call per question, repeated for each configured batch width over multiple measured runs. The retained historical receipts measure wall time and token usage. They do not retain paired answers, so they do not establish that batching left the decisions unchanged. The sequential strategy re-sends the state once per question, while batching shares a round trip and state transmission.

5 shows the measured speedup per batch width and the total-token ratio in separate panels.

Selected historical batching: jev-latest (2026-09-16)

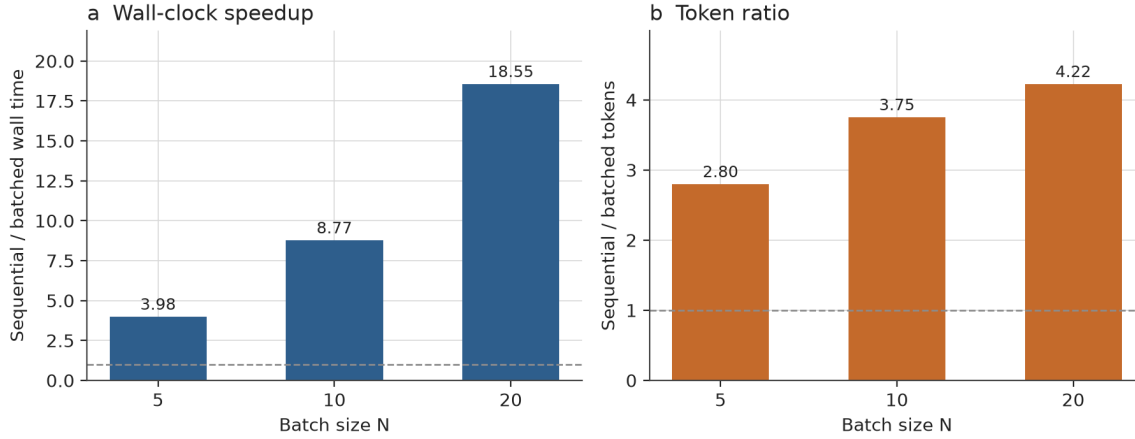


Figure 5: Batching speedup of the jev-latest model versus sequential single-question calls, measured by benchmarks/bench_batching.py on 2026-09-16 against the live API. Bars give the wall-time speedup of one batched call over one sequential call per question for each configured batch width (5, 10, and 20 questions per batch, tabulated in 1); a separate panel shows the total-token ratio, sequential tokens divided by batched tokens. The sequential strategy re-sends the state paragraph once per question. Each bar is annotated with its measured value, and the rendered title carries the model identifier and run date read from the benchmark JSON itself. The takeaway: the speedup grows with batch width as the fixed per-call overhead amortizes, while the batched strategy uses fewer total tokens in this recorded workload.

1 tabulates the measured speedups. The speedup grows with batch width, as expected from the round-trip accounting: the fixed per-call overhead is amortized over more questions, and the state is transmitted once rather than once per question.

Table 1: Wall-time speedup of one batched call versus one sequential call per question, per configured batch width, recorded by `benchmarks/bench_batching.py` against the `jev-latest` model on 2026-09-16. Values are injected from the benchmark JSON at render time.

Questions per batch	Wall-time speedup vs sequential
5	3.98
10	8.77
20	18.55

Recorded total-token use moves in the same direction. The recorded `token_cost_ratio` expresses the sequential strategy’s token consumption relative to the batched strategy’s: at the widest configured batch width the sequential strategy consumes 4.22 times the tokens of the batched call. This ratio measures total tokens, not reported USD. Input/output prices, provider surcharges and retries can differ; these historical receipts do not establish monetary savings.

6.2 Historical decision-pipeline latency

The second benchmark measures end-to-end latency of two composition pipelines — the network call plus the local composition logic, exactly as an application would run them:

- **composite_score** — one score call, then `composite_score` over the answer, then a `confidence_gate` verdict;
- **intent_routing** — one choice call, then `route()` dispatching to a trivial handler.

Each pipeline is executed 6 times; 2 reports the median (p50) and tail (p95) wall times per pipeline, and 6 plots them side by side.

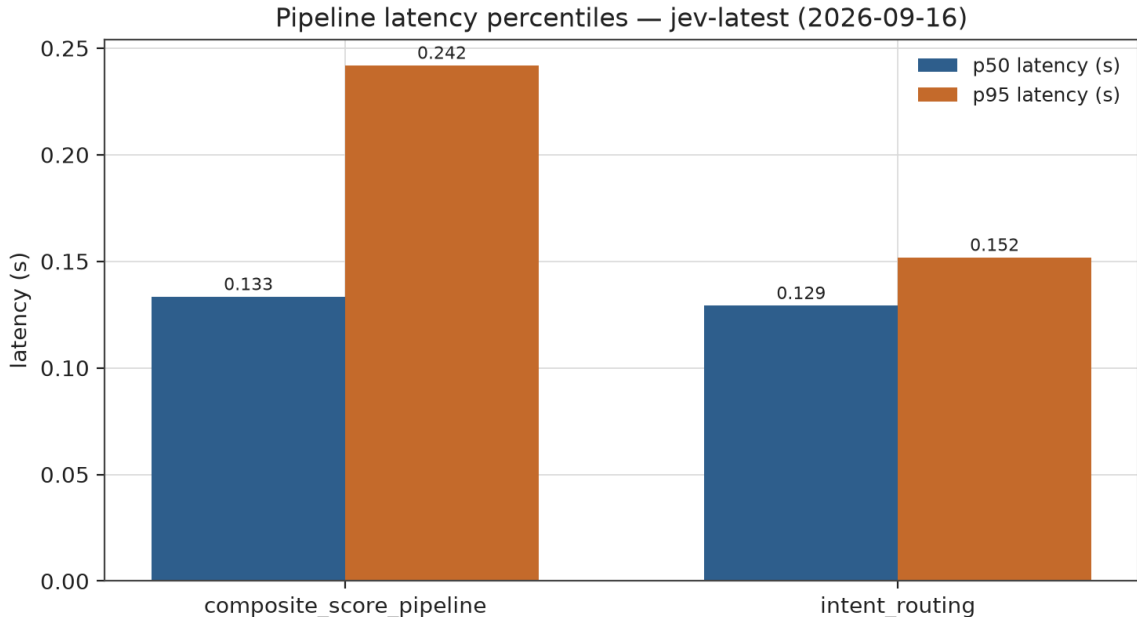


Figure 6: Median (p50) and tail (p95) end-to-end wall time per decision pipeline, measured by `benchmarks/bench_patterns.py` over 6 runs against the `jev-latest` model on 2026-09-16. Each pipeline group — composite scoring (one score call, then `composite_score` and a `confidence_gate` verdict) and intent routing (one choice call, then `route()` dispatch) — shows paired p50/p95 bars covering the full network round trip plus the local composition logic; values are tabulated in 2 and read from the benchmark JSON fields at figure-generation time. The bars describe these observed sessions. No isolated transport or composition baseline was recorded, so the figure does not estimate composition overhead or establish a service-level latency guarantee.

Table 2: End-to-end wall time (network round trip plus local composition logic) per decision pipeline, median and tail over 6 runs recorded by `benchmarks/bench_patterns.py` against the `jev-latest` model on 2026-09-16. Values are injected from the benchmark JSON at render time.

Pipeline	Median wall time, p50 (s)	Tail wall time, p95 (s)
<code>composite_score</code>	0.133	0.242
<code>intent_routing</code>	0.129	0.152

6.3 Historical proxy-agreement and repeat stability

The third historical benchmark compares reported confidence with repeat agreement, rather than measuring decision correctness against an independent label. `benchmarks/bench_calibration.py` runs 6 short states 5 times each against the `jev-latest` model and scores choice answers under a *self-consistency proxy*: a sample counts as correct when it agrees with the modal choice across the repeats of the same state. Because a yes/no probability carries no per-sample label to agree with, noul answers are scored for repeat-to-repeat stability instead, via the mean absolute gap between every pair of repeated answers. No external ground truth is consulted anywhere in this benchmark — the correctness proxy is self-agreement, not verified outcomes.

3 reports the aggregate scores over the run, and 7 plots the reliability curve per confidence bucket.

Table 3: Calibration summary over 6 states $\times$ 5 repeats recorded by `benchmarks/bench_calibration.py` against the `jev-latest` model on 2026-09-16. Choice correctness is a self-consistency proxy — agreement with the modal choice across repeats — not accuracy against ground truth; the mean pairwise noul gap measures repeat-to-repeat answer stability rather than correctness. Values are injected from the benchmark JSON at render time.

Metric	Value
Expected calibration error (choice, proxy)	0.0730
Brier score (choice, proxy)	0.0252
Mean pairwise noul gap	0.0050

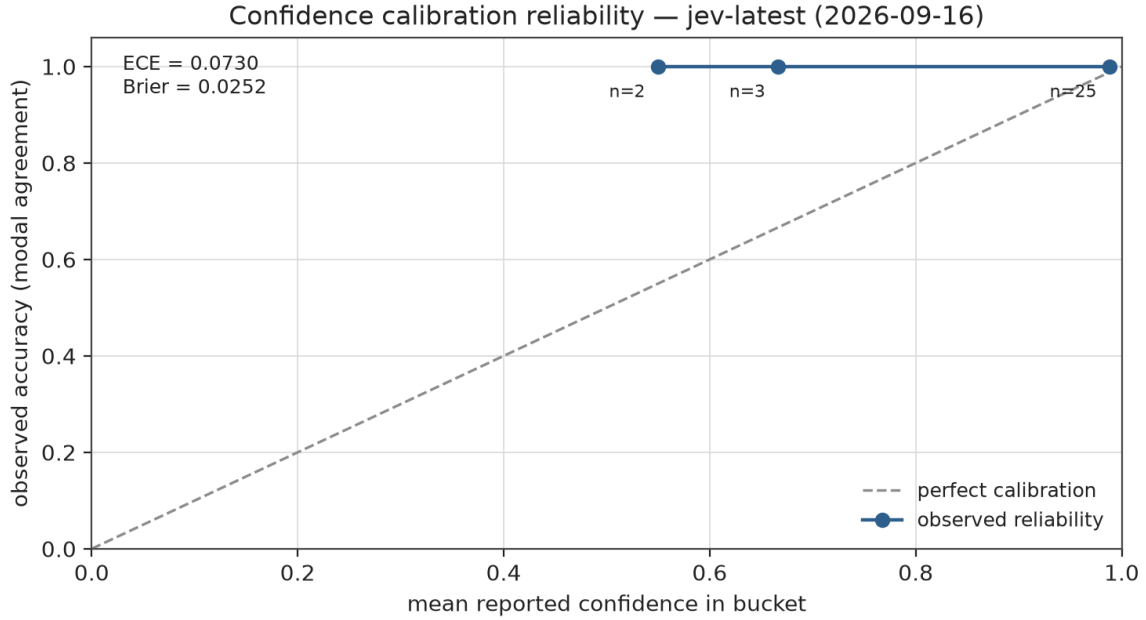


Figure 7: Reliability diagram of choice-answer confidence under the self-consistency proxy, measured by `benchmarks/bench_calibration.py` over 6 states $\times$ 5 repeats against the `jev-latest` model on 2026-09-16. Each point is a confidence bucket plotting mean reported confidence against proxy accuracy, where proxy accuracy is agreement with the modal choice across repeats of the same state; the dashed diagonal marks perfect agreement between stated confidence and observed proxy accuracy. Because the proxy scores self-agreement rather than verified correctness, proximity to the diagonal indicates consistency between stated confidence and repeat-stable behaviour — not truthfulness about the world. Aggregate scores are tabulated in 3.

The expected calibration error of 0.0730 and the Brier score of 0.0252 describe reported choice confidence against modal self-agreement labels. The mean pairwise noul gap of 0.0050 describes repeat-to-repeat answer variation on the same states. These observations are limited to the retained workload and its self-consistency proxy. They do not establish probability calibration against independent ground truth, confidence ordering or internal coherence, and they do not determine a necessary or sufficient condition for deploying decision gates.

6.4 Interpretation of the historical receipts

The retained observations support narrower conclusions. Batching reduced wall time and total-token use for this workload and requested model alias; paired answer equivalence and USD savings remain unmeasured. Pattern latency includes transport and composition together, so composition overhead cannot be isolated. Reported confidence and noul stability describe repeated self-agreement under the stated proxy, which can coexist with incorrect decisions. Ground-truth calibration, cross-model transfer and production routing quality require independent labeled evaluation.

6.5 Completed CPU comparator studies

The selected offline comparison comprises 9 serial jobs with 93046 planned cells: 68894 completed and 24152 unsupported. It retains 68870 completed quality cells separately from other cell roles. Fixed training-frequency references, TF-IDF logistic re-

gression and histogram gradient boosting provide reproducible comparators. Unsupported text/numeric task combinations remain visible; these jobs made no model-provider HTTP requests. Absence of API billing does not quantify local compute expense.

On the canonical final-train test views, fixed TF-IDF logistic regression achieves 87.69% accuracy on BANKING77 and 79.84% on CLINC150/OOS. These are supervised comparator results, not measurements of Jev, Kev, Jeff or a hosted chat model. Official tests had prior exposure in the research workflow; fixed settings and retained provenance make the evaluation auditable but do not create a new untouched holdout. CLINC aggregate accuracy is accompanied by a separate OOS detection audit; good overall intent classification does not establish reliable rejection of unseen intents.

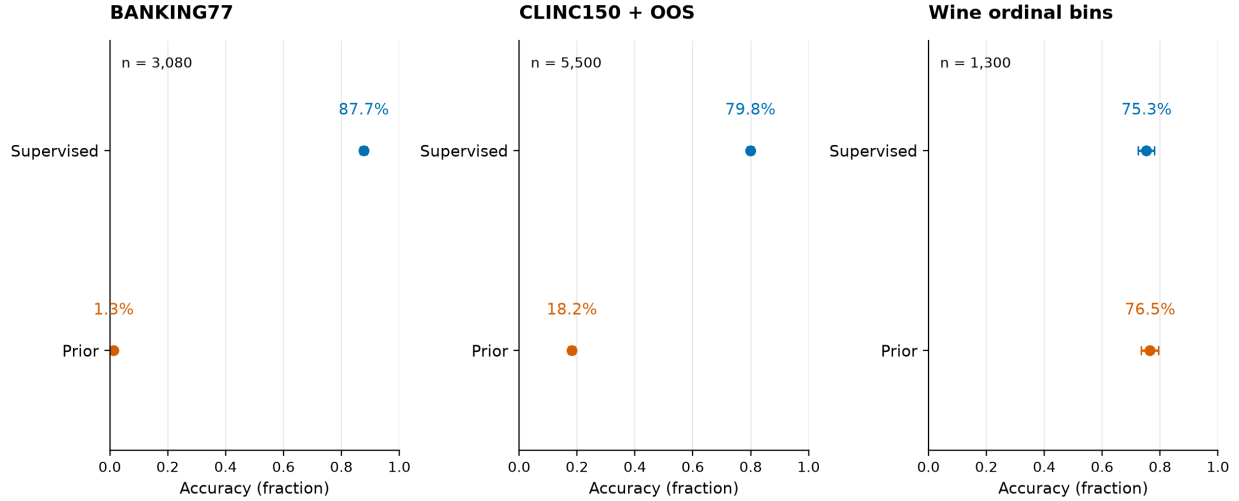


Figure 8: Canonical official-test accuracy of the applicable fixed CPU comparators after final train-plus-validation refitting, with retained grouped percentile intervals. Points and interval whiskers compare training-frequency references with the configured text classifier or numeric classifier; inapplicable task/backend pairs remain unsupported in the report. Intervals describe fixed evaluated predictions, not refitting uncertainty. Different datasets have different vocabularies and class distributions; their accuracies are not a common difficulty scale.

On Wine, the numeric comparator’s raw bin-index MAE is 0.2770, compared with 0.3319 for the training-frequency reference. Classification accuracy and scalar error need not rank these models identically: an argmax can favor a common category while its probability-weighted score is farther from the target index. The bins change the original grade target; these errors cannot be compared directly with published errors on the original grade scale [Cortez et al., 2009].

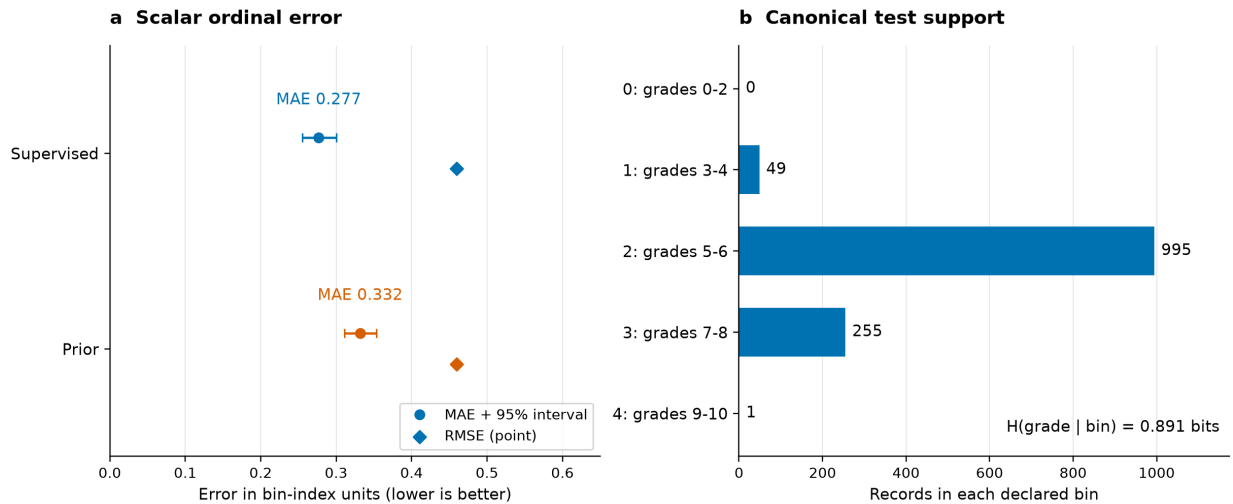


Figure 9: Wine scalar MAE and RMSE in declared bin-index units for the canonical final-train numeric comparator and training-frequency reference. The MAE interval uses the retained grouped resamples; RMSE is a descriptive point estimate. Classification accuracy uses genuine probability argmax and answers a different question. The lowest bin has no source observations, while the highest bin is sparse; full-vocabulary macro F1 retains absent classes, and these observations do not establish extreme-grade performance.

6.6 Cohort sensitivity and training-fold validation

The leakage-clean sensitivity removes view-specific input-group overlaps and conflicting labels before a separate fit. It changes both the fitted data and, where exclusions apply, the evaluated cohort. Canonical and cleaned metrics therefore describe distinct treatments. Their differences do not isolate a causal effect of leakage, and separate confidence intervals are not a paired test of that effect. The canonical benchmark remains available as published; the sensitivity makes the exclusion convention inspectable.

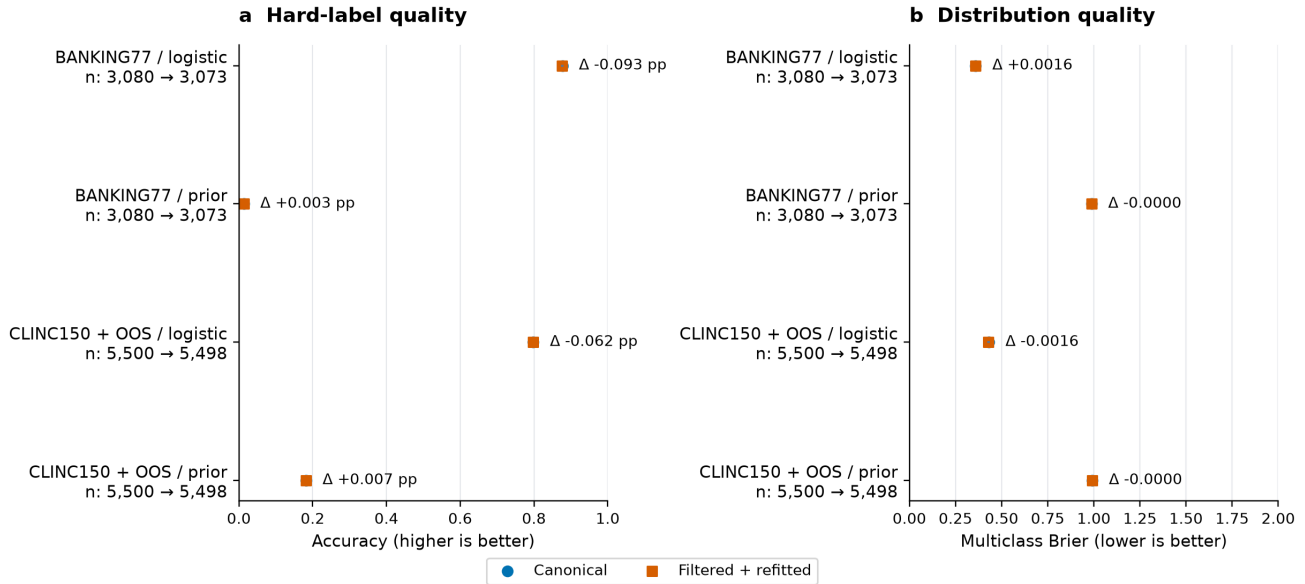


Figure 10: Canonical versus separately refitted leakage-clean accuracy and Brier loss for applicable CPU comparators, with evaluated group or row counts and descriptive metric differences disclosed. Cleaning can change training examples and test membership, so the contrast is a cohort-and-refit sensitivity rather than a paired causal comparison. Brier losses use the declared full-vocabulary convention and the same probability meanings within each comparator; lower values do not establish calibrated posteriors.

BANKING77 training-fold rotations evaluate every original training example once as validation, while keeping the official test outside those selection jobs. The pooled out-of-fold interval resamples retained input groups with predictions fixed. Shared training examples across folds make model errors dependent; the interval excludes the variability of rerunning the fitting procedure [Bengio and Grandvalet, 2004]. Neither the fold plot nor pooled interval is evidence for transferring a validation-selected gate to the later final refit.

Validation gates expose a further quality/coverage tradeoff. A stricter accepted subset can have lower observed error but less coverage; a policy selecting no eligible groups has zero acceptance and unavailable risk. The plotted Wilson bounds belong to recorded validation candidate selection. Adaptive threshold scanning and limited group counts prevent interpreting them as simultaneous deployment guarantees [Geifman and El-Yaniv, 2017]. No temperature scaling or other post-hoc probability recalibration is claimed by these gate results.

6.7 Partial native execution and a halted hosted probe

The selected later native sensitivity study retains 2005 completed cells, alongside unsupported, failed, unresolved and future unattempted work. Its compact cohort and source differ from the full CPU comparison. Resource interruptions, inherited wall allowances and unknown operational windows prevented full execution. Completed native predictions remain descriptive evidence for their actual questions and settings; they do not establish completion of the proposed full matrix or support pooling different model cohorts into an unqualified ranking.

The frozen hosted pilot has 18065 planned cells and made 1 actual HTTP attempt. It retains 1 failed, 19 unsupported and 18045 unattempted cells. The native Decisions capability probe returned a not-found response, with no resolved model/provider, usable prediction, tokens or charge available. No hosted quality, warm-repeat or graphical experiment ran. A separately observed public metadata response is not a successful decision request and does not identify the original failure cause.

The shared hosted allocation is USD 25; 1 unknown-billing attempt stops further admission. Reported and reserved zero totals are bookkeeping observations, not proof of zero actual charge. The frozen pilot uses a serialized hosted request treatment; a higher general concurrency default would be a different latency/throughput experiment.

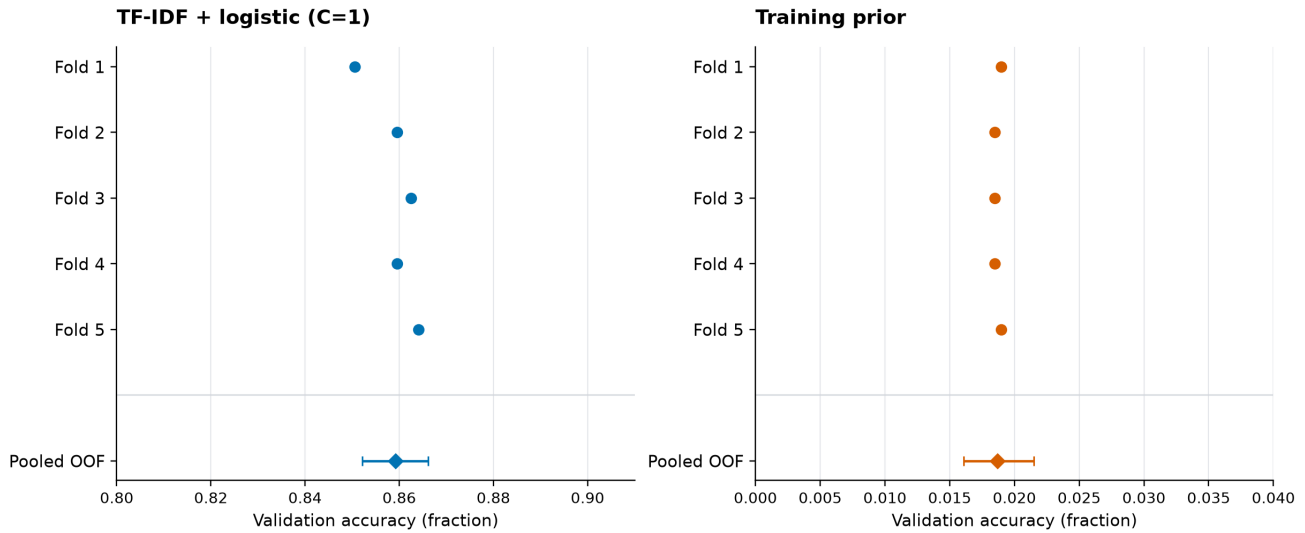


Figure 11: BANKING77 training-fold validation accuracy for the fixed classifier and training-frequency reference, with the pooled out-of-fold point estimate and its retained grouped percentile interval shown separately. Each original training row contributes a validation prediction once; fitted models have overlapping training sets. The interval concerns fixed pooled predictions and does not account for correlated training/refit uncertainty or establish a universal cross-validation variance estimate.

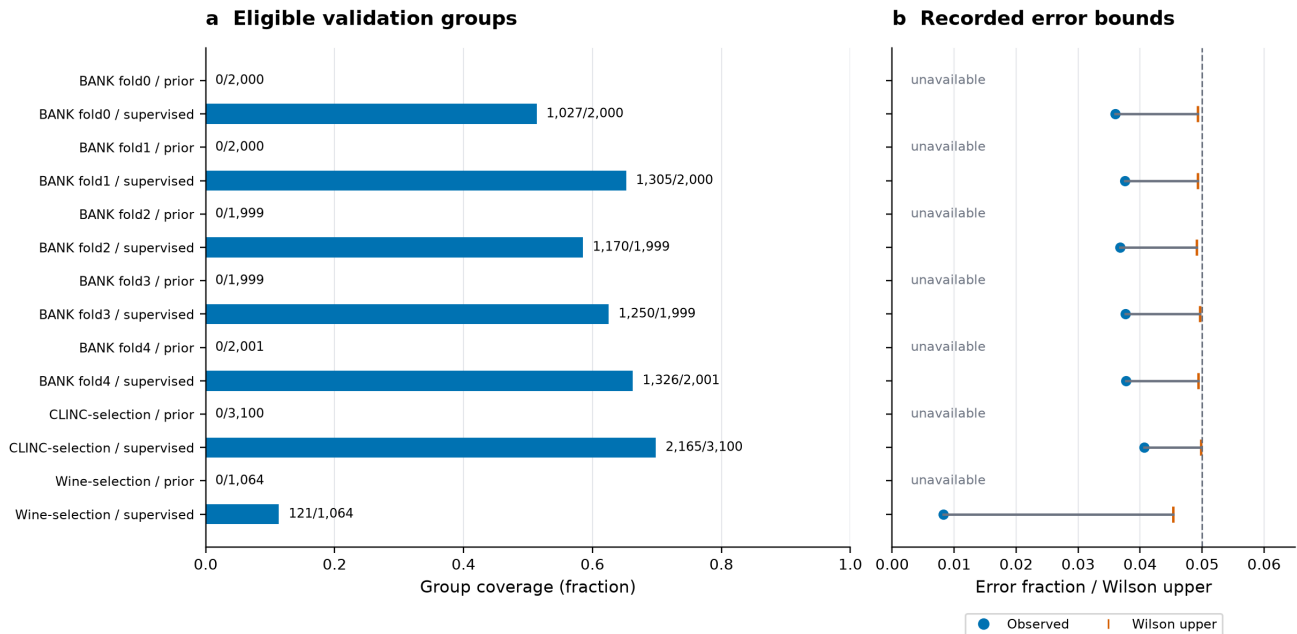


Figure 12: Recorded validation gate coverage, observed accepted-group risk and Wilson upper bound for the fixed selection models. Group correctness requires all accepted decisions in the group to be correct. A gate with no eligible threshold is displayed as zero coverage with unavailable risk and bound, not perfect performance. Thresholds were selected adaptively on validation data; these bounds are descriptive and do not carry a simultaneous risk guarantee or transfer to final refits.

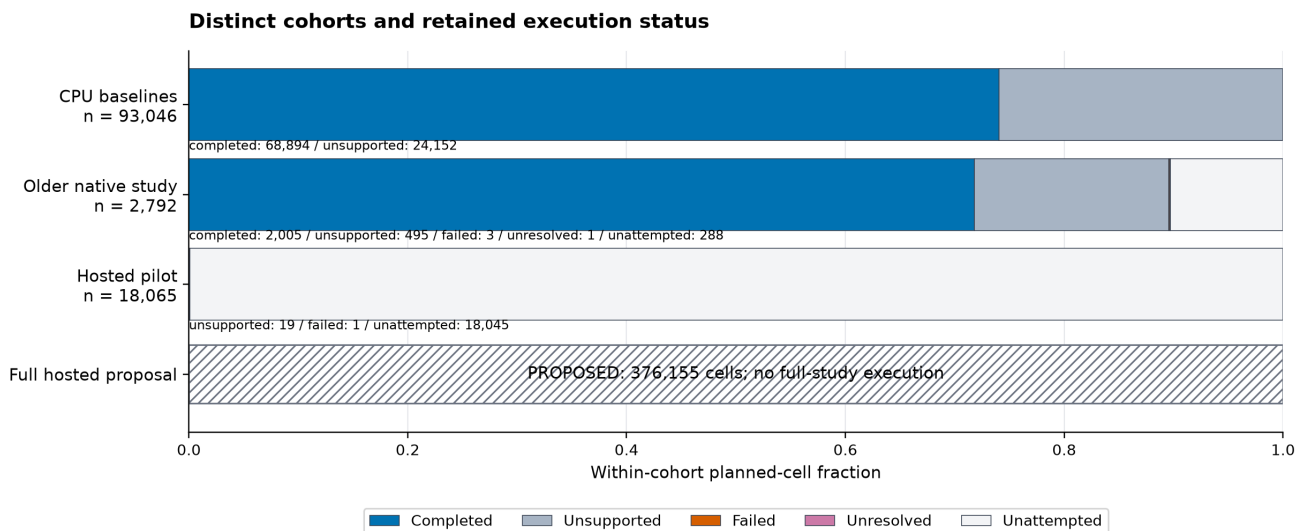


Figure 13: Planned execution-status coverage for the selected completed CPU studies, partial native sensitivity study and halted hosted pilot. Completed, unsupported, failed, unresolved and unattempted cells retain their distinct meanings and full denominators. A started cell without a terminal outcome is unresolved. The panels bind different cohorts and source identities and compare execution coverage, not model accuracy or economic efficiency; the hatched full-hosted proposal remains unexecuted.

The empirical contribution is consequently bounded: reproducible CPU references, retained historical API observations and inspectable failures of broader execution. The new plots make probability interpretation, cohort changes, selection limits and missing execution visible. Successful software verification supports the implementation contract; it cannot fill an absent model-quality or billing observation.

7 Experimental Setup: Historical Receipts, Environment, and Data Provenance

The results in 6 combine separately selected historical API receipts, CPU studies and partial execution reports. This section distinguishes their protocols, data roles and identities from the current software/render environment. Reproduction preserves intent and provenance; it does not promise identical responses from a changing remote service.

7.1 Software and documentation identities

The current package/render edition is daf-jev 0.6.0, generated under 3.14.4 on macOS-27.0.1-arm64-arm-64bit-Mach-0. Those values describe variable generation, not the historical benchmark execution environment. The old receipts lack a complete source, Python, hardware and dependency identity; those historical fields remain unknown. The package targets Python 3.10 or newer. Core runtime dependencies are httpx and pyyaml; optional supervised benchmarking uses scikit-learn and figure generation uses matplotlib/Pillow.

The model-level documentation claims in 3 are grounded in a retained hash-manifested TypeSafe snapshot (708902db9820d9d8, 111 pages). That identifies source documentation bytes, not the model's weights or current hosted behavior.

7.2 Historical benchmark configuration

The batching receipt compares one batched call with sequential single-question calls over a fixed synthetic paragraph and a mixed noul/choice/score question set. The retained widths are 5, 10 and 20. It records timings and token totals; answer equivalence and monetary savings were not independently measured.

The pattern receipt runs composite scoring and intent routing 6 times and records end-to-end wall time plus usage. It includes both network and composition work, without a separate overhead baseline. The calibration receipt runs 6 inline states 5 times against jev-latest, using modal self-agreement as its correctness proxy and pairwise noul gaps as stability. None of these inline states forms an independent ground-truth test corpus.

The command scripts expose their protocol through explicit arguments. manuscript/config.yaml records manuscript experiment settings; a render setting is not proof that a historical script consumed it. Actual run counts and settings must come from

the selected receipt, with discrepancies rejected or explained. Legacy scripts resolve a key through their documented environment/.env chain and skip without a key; they are outside the offline test suite.

7.3 Measurement and evidence selection

Historical wall times include transport, parsing and composition; retries and network conditions can influence them. They are observations, not conservative guarantees for future application latency. Token totals are response usage, not dollar charges. Where an old receipt does not declare its percentile estimator, the current nearest-rank helper cannot retroactively determine it.

The selected result files and their hashes bind prose and figures to one reviewed evidence set. The reproducible build timestamp is 2026-10-08T19:32:43Z; this pinned timestamp is neither the actual render time nor a benchmark execution date. Independently choosing the newest file for each benchmark cannot establish a common model/date/environment. Retained receipts remain unchanged when new experiments run.

7.4 Dataset views and evaluation roles

The new run format records exact prepared dataset bytes, source/config hashes, explicit model profiles, probability/confidence provenance, seed, cell order, limits and every attempted request. Train fits baselines; validation selects policy thresholds; test evaluates the frozen decision rule. Prior test exposure remains disclosed and is not undone by this role assignment. Repeated timing cells retain their leakage group and are separated from initial quality cells. Hosted spend is admitted against a sourced conservative liability, then reconciled with reported charges; unknown cost remains unresolved. Local monetary expense requires additional rate/hardware measurement.

The detailed executable protocol is docs/decision_benchmarking.md. It supports synthetic controls and pinned BANKING77, CLINC150/OOS and Wine preparation. Catalog discovery is not hosted inference, adapter tests are not local weight execution, and offline policy replay is not executed end-to-end latency. The selected reports establish only the executions they retain; historical measurements do not establish acceptance of a later lane.

A later frozen Mac study exposed an expanded per-dataset timing selection instead of the requested globally bounded shared cohort. Execution stopped at a clean boundary; completed observations and future unattempted denominators remain in their original experimental identity. Corrected sampling and ordered prompt controls require a new prospective source/cohort and fresh input proofs. The declared task mixture must follow the protocol before predictions, rather than observed test accuracy or model success. Prior preparation, execution and teardown still consume the same cumulative profile allowance.

Probability origin, numerical validity, meaning and calibration are distinct evidence. Native compatibility scores, discriminative class estimates, training frequencies and analytical conditional beliefs must retain their own interpretations; unknown meaning stays unknown. Proper scores describe the supplied rows against the stated target, without establishing calibrated posteriors or empirical CPTs. Earlier synthetic variants are not matched causal permutations, and sorted chat presentation with rotated schema enums does not measure prompt-position sensitivity.

The real tasks come from primary public datasets. BANKING77 supplies fine-grained banking intents [Casanueva et al., 2020]; CLINC supplies intent classification with explicit out-of-scope examples [Larson et al., 2019]. Their official test splits are retained. Wine combines red and white physicochemical observations with an explicit wine-type feature [Cortez et al., 2009]. The benchmark projects original quality grades into ordered bins and reports the source-grade distribution and information loss separately. This projection is a new evaluation target, not a reproduction of the original paper’s regression results.

Frozen fold packs bind complete original row inventories, grouping, assignments, algorithm, seed and preparation dependencies. Selection views omit the designated test split. Final-train views refit using the non-test training and validation data, retain test scoring, and contain no validation pool for transferring a policy fitted earlier. The protocol provides BANK training-fold rotations for fixed published HTTP models or fitted comparators on declared validation IDs; the selected completed rotations are CPU comparator jobs. An HTTP model is not retrained merely because its inputs carry fold metadata.

Canonical and leakage-clean views remain distinct. Input-group overlap and conflicting-label exclusion are view-specific, applied before a new fit when evaluating a fitted comparator. The exclusion rule can use labels; it is disclosed as a sensitivity analysis rather than a label-blind sampling procedure. It addresses overlap within these prepared splits, not possible overlap with a pretrained model’s undisclosed training corpus.

7.5 Synthetic controls and the sampling unit

Seeded synthetic tasks cover binary rules, categorical decisions, ordinal rubrics and analytical Bayes questions. Their state fields and generating rules define known labels or soft targets. Distractor, negation and context-length variants stress the parser/model

contract, but an unmatched variant collection is not a causal experiment on those factors. Binary base-task prevalence follows its independent generating conditions rather than forced class balance.

Matched cyclic Choice controls keep state, instructions, semantic options, truth, split and input group fixed while rotating option order. Complete groups remain together, and the loader validates actual semantic and order hashes. They are quality controls and do not inflate the timing cohort. For chat, prompt order and schema enumeration vary jointly under the declared serializer; this treatment cannot isolate either mechanism alone. Earlier sorted chat prompts and earlier inert rotation metadata are preserved as limitations of their original studies.

The global timing pack selects ordinary test input groups deterministically from the ordered supplied pools, balancing declared task/dataset participation without reading targets. This target-free statement applies to the timing selector only. Upstream pilot pools were class- or grade-stratified using labels, so the entire pipeline is not outcome-blind. One physical example represents each selected ordinary group; repeated timing observations keep the same complete state and question vocabulary. Full-group matched controls and validation probes have separate identities and denominators.

7.6 Timing, resource and economic treatments

Quality passes precede warm-repeat passes. A new serializer, probability contract or cohort requires a fresh primary under the new source; earlier primaries cannot serve as identical-input repetitions. Frozen cyclic profile schedules distribute repeat positions, but small repetition counts and sequential serving do not eliminate thermal, network or background-load effects. Reported request latency, SDK resource windows, startup/load time, owned cleanup time and whole-profile cumulative allowance are separately scoped.

Local execution admits only a verified owned runtime and complete input within a frozen operational bound. A conservative complete-prompt upper bound is labeled differently from an exact tokenizer count. An operational hardware bound is not the model's advertised context limit. Sampled process RSS and an OS physical-footprint observation are different resource measures; neither establishes continuous Metal memory peaks. Primary-only diagnostic observers can affect latency, and no unmeasured correction is subtracted.

A hosted plan freezes endpoint, requested model, provider preferences, schema, token limits, tariffs, budget and concurrency before predictions. Probes, failures and retries consume the same allocation as quality calls. Paid native admission cannot use one catalog context length as an unverified aggregate billable-input ceiling; zero-priced catalog candidates still require response and billing reconciliation. A failed capability probe is retained without endpoint substitution or a new budget ledger. These restrictions can leave a large part of a frozen study unattempted, as the selected coverage figure shows.

8 Reproducibility: Selected Evidence, Hashed Inputs, and Rendered Artifacts

Measured values enter the manuscript through double-brace placeholders. Exact selected receipts, the documentation snapshot, source identities, variables and figure registry form the reproducibility chain. A rendered artifact verifies consistency with retained evidence; it does not supply missing historical provenance or prove a new model execution.

8.1 Artifacts

Figures are generated by `src/daf_jev/figures.py` into `output/figures/`, with `figure_registry.json`. The legacy default registry remains available; this manuscript additionally opts into the selected empirical study figures. Schematic architecture/primitives/confidence figures need no measurements. Batching, latency and calibration figures consume the selected historical receipts. The legacy default graphical abstract also uses those historical receipts; this edition's opt-in study overview instead consumes the selected CPU and hosted summaries, including the separately bound older native status. Missing or incompatible evidence must fail rather than fabricate a result.

```
uv run python scripts/generate_figures.py --include-study
uv run python scripts/z_generate_manuscript_variables.py
uv run python scripts/render_pdf.py \
    --output output/pdf/reproduction.pdf \
    --artifacts-dir output/reports/reproduction-build
```

The variable generator writes `output/data/manuscript_variables.json` from package/configuration metadata, test evidence, documentation manifest and selected benchmark JSONs. Strict mode rejects missing required inputs; `--allow-draft` substitutes draft sentinels and is not final evidence. The standalone PDF renderer resolves tokens directly from the map, checks unresolved citations/references/images and writes the selected fresh PDF path, refusing to overwrite existing outputs. The exclusive artifacts

directory retains combined Markdown, TeX, bibliography and compiler diagnostics. An overfull vertical box fails rendering because missing or clipped prose can evade a token-only check. External-template substituted sections are an optional integration. Inspect the final PDF before publication; root installation and public release are separate actions.

The retained TypeSafe snapshot comprises 111 pages (1.1 MiB), scraped on 2026-09-24 and identified by 708902db9820d9d8. Offline verification rehashes those exact bytes:

```
uv run daf-jev docs-verify
uv run python scripts/scrape_docs.py --check --manifest docs/reference/MANIFEST.json
```

Omitting `--manifest` from the scraper's check performs a fresh network comparison. A successful retained-byte verification is not proof that the current upstream website or hosted model is unchanged.

8.2 Test evidence

Tests follow the no-mock convention: real transports connect to an ephemeral local HTTP fixture, including persistent connection, error, timeout and cancellation paths. Live tests use an explicitly supplied process-environment key; they skip without it. Current counts are 56 files, 1590 unit tests and 2 live tests, subject to the completed evidence used for this rendered edition.

```
uv run pytest tests/unit --cov=src
```

Coverage is 93.11% under the repository's configured gate. This edition selects a retained native verification record binding coverage JSON, JUnit outcomes, live-test collection and equal before/after/current input inventories. Its regeneration consumes those hash-verified exports without executing live tests, recollecting counts or requiring ephemeral raw coverage. Malformed or stale explicit selection fails rather than falling back to another source. Collection counts and completed runtime coverage are different observations; neither can be inferred from a previous tree. Skipped live tests do not establish live acceptance.

8.3 Run custody and scientific gates

The new benchmark format freezes source/config/catalog/dataset hashes, cohort, selected IDs and protocol in a unique run directory. Its append-only linked journal and independent saved head retain all attempts, failures and unresolved starts. Input/source identity is checked before and after execution; report reduction rechecks inputs. Resume avoids automatic replay of uncertain paid attempts. Reported spend, unresolved liability and unknown local cost remain distinct.

Publication evidence must additionally establish genuine model execution, complete task vocabulary, independent held-out labels, frozen policies, adequate group counts, uncertainty and cost reconciliation. Requested aliases do not establish immutable weights; catalog discovery does not establish access. The historical calibration metric remains a self-consistency proxy. This edition is version 0.6.0, with reproducible build timestamp 2026-10-08T19:32:43Z; its render environment is macOS-27.0.1-arm64-arm-64bit-Mach-0 under 3.14.4. The pinned build timestamp does not record the actual render time. Release identity and archival publication need separate verification.

Protocol corrections must preserve the original experimental identity and its limitations. A clean admission stop is bound by an additive receipt and unchanged manifest/journal/head; it does not relabel future unattempted work as complete. A revised globally shared timing cohort or ordered prompt serializer needs a newly accepted source, selected-ID pack and complete input/custody proofs. Earlier observations cannot be retroactively converted into matched permutation evidence or identical-prompt repetitions, and a new manifest cannot reset a shared resource allowance.

8.4 Evidence tiers and portable reuse

A publication selection binds the historical benchmark receipts, current software-verification record and each empirical study summary separately. The figures and variable generator consume the same selected bytes. A reduction produced by newer software retains both its reducer identity and the older inference identity; successful reporting cannot retroactively upgrade the source under which a model ran. Standalone summaries retain aggregate data and public provenance while keeping prompts, credentials, personal paths and process-control receipts private.

Runtime reuse additionally requires pinned model and tokenizer files, serving code and consumed runtime/distribution identities, actual launch configuration, and owned listener/process evidence. Hashing before and after a consumption interval detects boundary changes but is not continuous file-access attestation or a proof linking every source line to the running binary. A copied report can be audited read-only; resuming its executor requires the original ownership and filesystem contract, and unresolved attempts are not silently replayed.

The current source manuscript, CPU report, historical native results and halted hosted probe remain separate artifacts. Their re-generation proves consistency with selected records. Scientific comparison additionally requires a common declared task/cohort, suitable labels and policy selection, supported uncertainty interpretation, adequate independent groups and reconciled costs. Publishing, archiving a new release and sending further model requests are separate authorized actions.

9 Related Work and Discussion: What a Decision Benchmark Can Establish

9.1 Structured generation and native decisions

Formal constraints can remove many syntactic failure modes from generated output [Willard and Louf, 2023]. Provider JSON-schema interfaces bring these methods to ordinary application calls, subject to endpoint-specific support [OpenRouter, 2026b]. Native decision interfaces instead expose typed judgments and often distributions directly. The distinction is useful for adapter design, but neither route makes semantics correct merely by construction. A controlled comparison must hold task, state, vocabulary, output interpretation and execution treatment fixed before attributing differences to the model interface.

TypeSafe’s RLCD description supplies a training motivation [TypeSafe AI, 2026f], without establishing a complete reproducible training recipe or an independently verified guarantee for all domains in the cited record. News coverage and independent commentary are retained for historical context [Claburn, 2026, Corvin, 2026, Morales, 2026]. They do not substantiate the package’s software architecture or a new provider’s capability. Complete model-weight provenance and empirical endpoint tests provide different evidence from launch documentation.

9.2 Probabilistic forecasts and selective action

Proper scoring rules evaluate probability forecasts through their relationship to outcomes [Brier, 1950, Gneiting and Raftery, 2007]. This prevents a common conflation: sharper forecasts can be more useful when reliable, but sharpness alone is not correctness calibration. Neural-network calibration research further shows why fitting and evaluating an uncertainty transformation requires appropriate held-out data [Guo et al., 2017]. The present comparator experiment records class estimates and training frequencies; it does not demonstrate post-hoc temperature calibration or a general calibrated-posterior interpretation.

Selective classification studies how a predictor can abstain while controlling error on the accepted subset [Geifman and El-Yaniv, 2017]. Its guarantee-producing procedures depend on their stated assumptions and threshold-selection construction. Our validation Wilson rule is a useful auditable policy selector, but adaptive scanning does not inherit those guarantees. A confidence score may rank answers without accurately estimating their probability of correctness. Deployment also needs the costs of rejected decisions, escalation capacity and shift monitoring; a high accepted accuracy at negligible coverage can be operationally unhelpful.

FrugalGPT studies learned cascades of differently priced language models [Chen et al., 2024]. Its cost/quality motivation applies to modular decision backends, but reported gains do not transfer to this implementation. A cascade must include weak-call cost, invoked strong calls, failures and retries, and validate the router on data separate from test evaluation. Replay can compare the recorded prediction pairs; it cannot establish end-to-end serving savings. The current hosted probe supplies no successful paid cascade comparison.

9.3 Multidimensional benchmarking and data overlap

HELM argues for broad evaluation across scenarios and metrics rather than one aggregate score [Liang et al., 2023]. The present framework follows that methodological direction for decision quality, probability semantics, selective coverage, execution reliability and cost. It does not implement every HELM dimension, establish fairness across protected groups, or claim compliance with another benchmark’s load-generation rules. A quality table restricted to successful arms is incomplete without the full planned status matrix.

BANKING77 and CLINC were designed for intent-recognition research, with CLINC explicitly addressing out-of-scope prediction [Casanueva et al., 2020, Larson et al., 2019]. These datasets offer public labels and reproducible task definitions, but the fixed bag-of-words comparator is not a reproduction of the papers’ model architectures. Wine supplies a different input modality and target geometry [Cortez et al., 2009]; the derived bin-index task must be interpreted on its own scale. Cross-task averages can obscure these differences and require a separately justified aggregation rule.

Within-corpus cleaning and pretrained-model contamination are different problems. The former can be audited from retained source rows and groups. The latter requires information about training exposure or a valid detection method. Oren et al. develop a contamination test based on exchangeable benchmark order and model likelihoods under specific assumptions [Oren et al., 2024]. Those queries and assumptions are not established for the decision adapters here. Public benchmark availability therefore

motivates an overlap caveat, not an accusation or proof that any selected model trained on a particular test set. Synthetic fixtures reduce dependence on memorized public labels for a specified mechanism, while offering limited ecological validity.

9.4 Graphical reasoning and information acquisition

Variable elimination gives exact inference for a specified finite factorization, with complexity governed by graph structure and elimination order [Dechter, 1999]. Model-supplied factors introduce a separate identification problem: the calculation can be exact while the factors are poor representations of the target process. The disclosed reference experiment makes this distinction measurable through factor and posterior discrepancies, but it is not causal discovery or a validation of arbitrary elicited world models.

Active data selection asks which observation is expected to be informative [MacKay, 1992]. The implemented maximum-marginal-entropy policy is deliberately simpler. It does not integrate over future observations to optimize expected information gain, and it does not maximize expected downstream utility per cost. Observation trajectories under a known reference oracle test the implementation and surrogate factors; field deployment would require defensible observation models, costs and decision loss.

9.5 Limits of the selected evidence

The historical API workload lacks complete weight, environment and paired-answer identity. The current CPU studies use fixed settings and disclosed prior official-test exposure; grouped intervals concern fixed predictions rather than refit uncertainty. The native study is partial and uses a different compact cohort. The hosted study halted after an unsuccessful probe with unresolved billing. No common all-provider quality, latency or cost-per-correct-decision ranking follows from combining them.

Execution failure is informative about the attempted interface and conditions, but missing execution is not model error or a fabricated negative prediction. Conversely, success on a small validation probe does not establish maximum vocabulary, context capacity or reliability. Resource guards provide sampled operational evidence, with unknown memory and custody intervals retained. Conservative approved accounting conventions cannot repair missing observations or turn an unresolved window into measured elapsed time.

The package and this manuscript support prospective studies: freeze the tasks and profiles, validate full input consumption, separate selection from scoring, reconcile all attempts, and report coverage alongside conditional quality. New source acceptance establishes software behavior at that identity. New scientific and release claims require their own evidence; none is inferred from the existence of a reproducible renderer or a public archival identifier.

10 References

The bibliography distinguishes primary methodological papers and dataset publications from provider documentation and historical launch coverage. Proper scoring, calibration, selective prediction, cascades, exact inference and benchmark-overlap claims are attributed to their primary research sources. Documentation citations describe the recorded interface, not empirical acceptance of a present service. Historical access dates and snapshot notes remain attached to the original entries; newly checked sources carry their own access date. All citation keys resolve through `manuscript/references.bib` using the repository’s normal bibliography pipeline.

References

- Yoshua Bengio and Yves Grandvalet. No unbiased estimator of the variance of K-fold cross-validation. *Journal of Machine Learning Research*, 5:1089–1105, 2004. URL <https://www.jmlr.org/papers/v5/grandvalet04a.html>.
- Glenn W. Brier. Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1):1–3, 1950. doi: 10.1175/1520-0493(1950)078<0001:VOFEIT>2.0.CO;2.
- Iñigo Casanueva, Tadas Temčinas, Daniela Gerz, Matthew Henderson, and Ivan Vulić. Efficient intent detection with dual sentence encoders. In *Proceedings of the 2nd Workshop on Natural Language Processing for Conversational AI*, pages 38–45. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.nlp4convai-1.5. URL <https://aclanthology.org/2020.nlp4convai-1.5/>.
- Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *Transactions on Machine Learning Research*, 2024. URL https://lingjiaochen.com/papers/2024_FrugalGPT_TMLR.pdf. Author-hosted published article; original preprint arXiv:2305.05176.

- Thomas Claburn. Typesafe ai debuts model for machines that plays doom, 9 2026. URL <https://www.theregister.com/ai-and-ml/2026/09/16/typesafe-ai-debuts-model-for-machines-that-plays-doom/5296711>. Independent coverage: USD 40M funding, latency/pricing corroboration, Jevons-paradox naming, and skepticism about the "hallucination-free" framing.
- Paulo Cortez, António Cerdeira, Fernando Almeida, Telmo Matos, and José Reis. Modeling wine preferences by data mining from physicochemical properties. *Decision Support Systems*, 47(4):547–553, 2009. doi: 10.1016/j.dss.2009.05.016. URL <https://www.sciencedirect.com/science/article/pii/S0167923609001377>.
- Magnus Corvin. Jev refuses to write a single word: What typesafe ai’s decision model does, and what nobody has verified yet, 9 2026. URL <https://www.orcarouter.ai/blog/jev-typesafe-system-one-what-we-know>. Independent analysis sorting vendor-claimed vs independently measured numbers; recounts Every’s tests; proposes the diagonal calibration-curve audit method.
- Rina Dechter. Bucket elimination: A unifying framework for reasoning. *Artificial Intelligence*, 113(1–2):41–85, 1999. doi: 10.1016/S0004-3702(99)00059-4. URL <https://ics.uci.edu/~csp/r48b.pdf>.
- Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL <https://proceedings.neurips.cc/paper/2017/hash/4a8423d5e91fda00bb7e46540e2b0cf1-Abstract.html>.
- Tilmann Gneiting and Adrian E. Raftery. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477):359–378, 2007. doi: 10.1198/016214506000001437. URL <https://sites.stat.washington.edu/people/raftery/Research/PDF/Gneiting2007jasa.pdf>.
- Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1321–1330. PMLR, 2017. URL <https://proceedings.mlr.press/v70/guo17a.html>.
- Stefan Larson, Anish Mahendran, Joseph J. Peper, Christopher Clarke, Andrew Lee, Parker Hill, Jonathan K. Kummerfeld, Kevin Leach, Michael A. Laurenzano, Lingjia Tang, and Jason Mars. An evaluation dataset for intent classification and out-of-scope prediction. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, pages 1311–1316. Association for Computational Linguistics, 2019. doi: 10.18653/v1/D19-1131. URL <https://aclanthology.org/D19-1131/>.
- Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, et al. Holistic evaluation of language models. *Transactions on Machine Learning Research*, 2023. URL <https://openreview.net/pdf/1882e7aa18c29c4487d64455658f6498456bc0dc.pdf>. Published article; original preprint arXiv:2211.09110.
- David J. C. MacKay. Information-based objective functions for active data selection. *Neural Computation*, 4(4):590–604, 1992. doi: 10.1162/neco.1992.4.4.590. URL <https://authors.library.caltech.edu/records/efefp-2j353>.
- Andrés Morales. Typesafe ai’s jev: The model that decides without text, 9 2026. URL <https://elsolitario.org/en/2026/09/16/typesafe-ai-jev-structured-decision-model/>. Independent analysis: RLHF/RLVR-vs-RLCD framing, autoregressive-vs-parallel sampling explanation, launch timeline, and caveats on vendor-run benchmarks and pricing sustainability.
- OpenRouter. Submit a decisions request, 2026a. URL <https://openrouter.ai/docs/api/api-reference/alphadecisions/submit-a-decisions-request>. Documented native POST interface; documentation availability does not establish successful access or model execution.
- OpenRouter. Structured outputs, 2026b. URL <https://openrouter.ai/docs/guides/features/structured-outputs>. Schema support and enforcement are endpoint-specific; empirical capability verification is separate.
- Yonatan Oren, Nicole Meister, Niladri Chatterji, Faisal Ladhak, and Tatsunori B. Hashimoto. Proving test set contamination in black-box language models. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/46e624c244cff669223d488defd4e835-Abstract-Conference.html.
- TypeSafe AI. Api reference, 2026a. URL <https://docs.typesafe.ai/api.md>. HTTP API for POST /v1/systemone: request/response shapes, the three answer types, usage tokens, and error/retry semantics. Historical verification metadata: earlier local snapshot docs/reference/api.md (snapshot id b79c9cd6008489f1).
- TypeSafe AI. Confidence, 2026b. URL <https://docs.typesafe.ai/confidence.md>. How confidence is derived from the probability distribution; the three-band act/caution/escalate pattern; risk-scaled thresholds. Historical verification metadata: earlier local snapshot docs/reference/confidence.md (snapshot id b79c9cd6008489f1).

TypeSafe AI. Patterns, 2026c. URL <https://docs.typesafe.ai/patterns.md>. Composable patterns for building systems on System One models, e.g. confidence routing and composite scoring. Historical verification metadata: earlier local snapshot docs/reference/patterns/ (snapshot id b79c9cd6008489f1).

TypeSafe AI. Primitives (questions), 2026d. URL <https://docs.typesafe.ai/primitives.md>. The three question primitives, their typed answers, parallel isolated evaluation, and the 32,000-token (150,000-character) shared request budget. Historical verification metadata: earlier local snapshot docs/reference/primitives.md (snapshot id b79c9cd6008489f1).

TypeSafe AI. Introducing system one models & jev, 9 2026e. URL <https://typesafe.ai/blog/introducing-system-one-models-and-jev>. Launch post by Diogo Almeida announcing System One models, Jev, the parallel-sampler architecture, RLCD training, latency/cost envelope, and early access. Canonical key: typesafe2026systemone.

TypeSafe AI. System one, 2026f. URL <https://docs.typesafe.ai/concepts/system-one.md>. Historical verification metadata: earlier local snapshot docs/reference/concepts/system-one.md (snapshot id b79c9cd6008489f1, scraped 2026-09-16T21:19:06Z).

Brandon T. Willard and Rémi Louf. Efficient guided generation for large language models, 2023. URL <https://arxiv.org/abs/2307.09702>.